The Essential of Chapter 6

Buffer management cut down to what the two mock exams ask: the cost of a miss, the `Buffers:` line, the victim, the log, the size of the pool

Advanced Data Management

2026-10-29

How to read this document

This is chapter 6 with everything removed that neither mock exam touches. Five sections, each ending in a *Mock drill* box that says which mock question lives there and the shape of the answer the rubric wants. Read it in order: every term is defined before it is used. Where a page of the full chapter is worth the detour, the section says so. Numbers to carry: RAM access 0.1 μs, SSD random read 100 μs, HDD random read 10 ms, sequential read 500 MB/s, page 8 KB; `orders` 3,185 pages, `customers` 73 pages.

1 The Pool and the Cost of a Miss

Concept: Buffer pool, frame, hit, miss

The **buffer pool** (`shared_buffers` in PostgreSQL) is a fixed array of page-sized **frames** in RAM plus a page table saying which disk page sits in which frame. Every operator asks the pool for a page, never the disk. If the page is in a frame the request is a **hit** and costs a RAM access; if not it is a **miss**: a frame is freed (a **victim** chosen by the **replacement policy**, written back first if it is **dirty**, that is modified since it was read) and the page is read from below. The **hit ratio** h is the fraction of requests that hit.

The pool pays off because of **locality**: pages used recently are used again (the root of every B^+-tree, the catalog, the hot customers), and pages near a used page are used next (the following page of a scan). A miss costs one page read, or a write plus a read when the victim is dirty, which is why a background writer keeps victims clean (§4).

Mechanism: Effective access time

If a hit costs T_{hit} and a miss T_{miss}, the average cost of a page access is

$$\text{EAT} = h\,T_{\text{hit}} + (1-h)\,T_{\text{miss}}.$$

Since T_{miss} is 10^3 (SSD) to 10^5 (HDD) times T_{hit}, the second term rules until $1-h$ is tiny: **think in miss ratios**. Going from 99% to 99.9% hits is not "one percent better"; it removes nine of every ten disk reads and divides the EAT by ten.

Worked example: The two numbers the exam expects

HDD, $h = 0.99$**:** $0.99 \times 0.1 + 0.01 \times 10{,}000 = 0.099 + 100 \approx 100$ μs. One miss in a hundred makes the average access a thousand times slower than a hit; a 99% cache on a disk is still disk-bound. To get EAT ≤ 1 μs solve $0.1h + 10{,}000(1-h) \leq 1$: $h \geq 0.99991$, fewer than one miss in ten thousand.
SSD, $h = 0.98$**:** $0.98 \times 0.1 + 0.02 \times 100 = 2.1$ μs, 21 times a RAM access. For EAT ≤ 1 μs: $99.9h \geq 99$, $h \geq 0.991$.

h	0.90	0.95	0.99	0.999	0.9999
EAT, HDD ($T_{\text{miss}} = 10$ ms)	1.0 ms	0.50 ms	100 μs	10.1 μs	1.1 μs
EAT, SSD ($T_{\text{miss}} = 100$ μs)	10.1 μs	5.1 μs	1.1 μs	0.20 μs	0.11 μs

Mock drill

Mock v2 B2(c): a B$^+$-tree lookup of 5 page reads where the root and the second level are always in the pool costs 2×0.1 μs $+3 \times 100$ μs ≈ 0.3 ms on SSD; on HDD the three misses alone are 30 ms, so the 1 ms target needs SSD, not more RAM. **Mock 1, 3.6**: given a hit ratio, compute the EAT, state it against the latency target, and say which miss ratio would meet it. Shape: formula, substitution with units, one sentence.

2 Reading the `Buffers:` Line

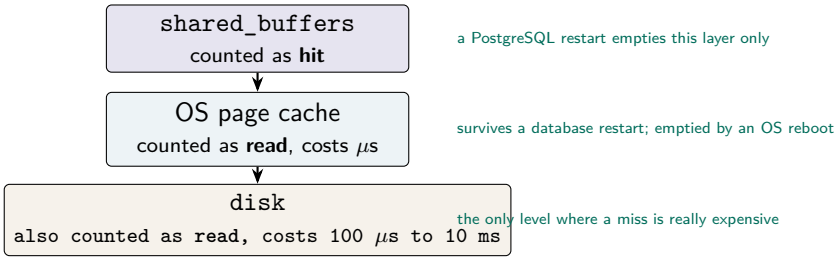
Concept: Anatomy of a `Buffers:` line

```
Buffers:  shared hit=40 read=3145 dirtied=2 written=1, temp
read=183 written=183
```

- `shared hit`: pages found in `shared_buffers`. No I/O.
- `shared read`: pages *not* in `shared_buffers`, requested from the operating system. They came from the **OS page cache** (microseconds) or from the disk (100 μs to 10 ms); the counter cannot tell which.
- `dirtied`: pages this query modified. `written`: dirty pages this query had to write itself to free a frame; nonzero means eviction is doing the background writer's job.

- `temp read/written`: temporary files of a sort or hash join that did not fit in `work_mem`. Not the pool at all: a chapter 4 symptom (`Batches` > 1, external merge).

Hit ratio of one execution: hit/(hit + read). The top node's line is the whole query; a node's line is that node's own pages.



Worked example: Mock v2 C3: 3,258, read to hit, and 1.3× rather than 1,000×

The December report ran in 82.9 ms right after a restart with `shared read=3258`, then in 62.3 ms with `shared hit=3258`.

3,258 = 3,185 + 73: every page of `orders` plus every page of `customers`, read once each by the two sequential scans; the join and the aggregate add no pages. Both tables are below `shared_buffers`/4 (§3.4), so no ring was used and all 3,258 pages stayed in the pool: the second run finds every one, `read` becomes `hit`.

Why only 1.3×? A restart empties `shared_buffers` but not the OS page cache, which still held the files. The 3,258 "reads" were served by the OS from RAM, at microseconds each: about 20 ms of system calls and copies, which is the whole gap. From disk, a sequential read of 3,258 × 8 KB ≈ 25 MB would add about 50 ms at 500 MB/s, and random HDD reads 3,258 × 10 ms ≈ 33 s. A 1.3× cold/warm gap is the signature of a pool miss that was an OS-cache hit.

Mock drill

Mock v2 A7: `read=3145` means 3,145 pages were not in shared buffers; some may come from the OS cache (answer b). Never "read from the disk". **Mock v2 C3** is the example above: three numbers (3,185 + 73; the hit ratio 0 then 1; the 20 ms gap against 33 s) and the sentence "read is not disk". Full chapter: §7.1–7.2, pages 16–17.

3 Choosing the Victim

3.1 LRU and FIFO

FIFO evicts the page *loaded* earliest, whatever happened since. **LRU** (least recently used) evicts the page whose *last access* is oldest: a hit renews the page. The exam asks you to run both on a short sequence and count faults.

Worked example: Three frames, requests A B C A B D A B

LRU (least recent on the left): A, B, C miss; A hit (B C A); B hit (C A B); D evicts C (A B D); A hit (B D A); B hit (D A B). **4 hits**, 4 misses. **FIFO** (oldest load on the left): A, B, C miss; A, B hit; D evicts A (B C D); A evicts B (C D A); B evicts C (D A B). **2 hits**. LRU wins because A and B are hot and keep renewing; FIFO ages them out regardless. Trace rule: under LRU a hit moves the page to the newest end; under FIFO a hit changes nothing.

Concept: Belady's anomaly and stack algorithms

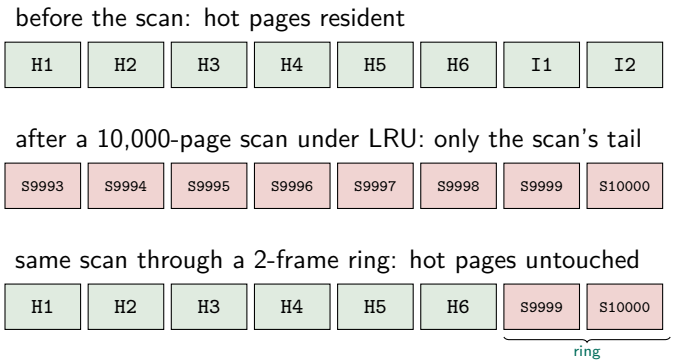
With FIFO, adding a frame can *increase* the faults: on 1 2 3 4 1 2 5 1 2 3 4 5, three frames give 9 faults and four frames give 10. LRU cannot do this: with n frames it holds exactly the n most recently used pages, which are among the $n+1$ most recently used, so every hit with n frames is also a hit with $n+1$ (a **stack algorithm**). On that sequence LRU gives 10 faults with three frames and 8 with four; the optimal policy (evict the page needed farthest in the future) gives 7, the yardstick nobody can beat.

3.2 Clock, and PostgreSQL's usage count

Exact LRU needs a list reordered on every hit, which with many cores becomes a lock. **Clock** keeps one **reference bit** per frame and a hand sweeping a circle: a hit sets the bit; to evict, the hand skips pinned frames, clears a bit that is 1 and moves on (second chance), and takes the first frame whose bit is 0. PostgreSQL's **clock-sweep** replaces the bit by a `usage_count` from 0 to 5: a hit increments it, the sweep decrements, a frame at 0 and unpinned is the victim. A page hit many times survives up to five sweeps; a page touched once survives at most one. That is LRU's behaviour at a fraction of the cost, and the reason the formula sheet says "clock-sweep, usage_count 0–5".

3.3 Sequential flooding

A large sequential scan reads every page once and never again. Under LRU each scan page arrives as "most recently used" and pushes out pages the rest of the workload reuses constantly. Once the scan has read more pages than the pool has frames, the whole hot set is gone and the next transactions reload it from disk one miss at a time: **sequential flooding**.



Worked example: The flooding exercise

Pool $N = 1{,}000$ frames, hot working set $W = 200$ pages, then a one-pass scan of $S = 10^7$ pages. **Hot pages surviving under LRU: 0**, since after the first 1,000 scan pages every frame holds a scan page and each hot page's last use is older. **Extra misses on the next loop: 200**, the whole hot set reloaded (2 s on HDD). **With a ring of $r = 32$ frames: all 200 survive**: the scan recycles its 32 frames (page k evicts page $k - 32$) and never touches the other $968 \geq 200$. The scan loses nothing, because a use-once scan would not have hit on its own pages anyway.

3.4 Ring buffers in PostgreSQL

Mechanism: Buffer access strategies

A sequential scan of a table **larger than `shared_buffers/4`** runs through a **ring** of 256 KB (32 frames) and cycles its pages there only. Smaller tables go through the main pool, because a table that fits comfortably is likely to be reused. Bulk writes (`COPY`) use a 16 MB ring; `VACUUM` a 256 KB one. Index scans never use rings.

Worked example: Is `orders` ringed?

`shared_buffers` = 128 MB = 16,384 frames; threshold 16,384/4 = 4,096 pages. `orders`, 3,185 pages, is below it: a `count(*)` goes through the main pool and leaves all 3,185 pages resident, which is why the mock's second run shows `hit=3185`. A 63,695-page archive is far above it: its scan occupies 32 frames and `orders` stays cached. At 64 MB the threshold is 2,048 pages and `orders` is ringed too: a second `count(*)` reads it again. The exam question is always "compare the table's pages to `shared_buffers`/4".

Mock drill

Mock 1, 1.3: trace LRU and FIFO on a short sequence, count hits, say which policy wins and why (hot pages renew under LRU). **Mock 1, 2.1**: define sequential flooding, give the 0 / 200 / 200 numbers, name

the ring buffer and its threshold. **Mock 1, 2.6 and 3.6**: decide from `shared_buffers`/4 whether a given scan is ringed, and what the second run's `Buffers` line will show. Full chapter: §4.2–4.3 pages 7–9, §5 pages 11–12.

4 Dirty Pages, Steal, Force and the Log

4.1 When a dirty page reaches disk

A write modifies a frame and sets its dirty bit; the disk is updated later, in one of three ways: **eviction** (the policy chose a dirty victim; the query that needed the frame pays a write to do a read, the `written` counter), the **background writer** (runs ahead of the clock hand so that victims are clean), and **checkpoints** (every 5 minutes by default, all dirty pages are written and a mark says "everything up to here is on disk", which bounds how much log recovery must replay).

4.2 Steal and force

Concept: Steal / no-steal, force / no-force

Steal: the pool *may* write a dirty page of an *uncommitted* transaction to disk, for instance to evict it. **No-steal**: it may not, so every dirty page of every running transaction stays pinned until commit. **Force**: at commit, *all* the transaction's dirty pages are written before commit is acknowledged. **No-force**: they need not be; commit only records the changes somewhere durable and cheap.

Each freedom creates a recovery need. If uncommitted changes can be on disk (steal) and the system crashes before commit, recovery must **undo** them, so it needs the old values. If committed changes can be only in memory (no-force) and the system crashes after commit, recovery must **redo** them, so it needs the new values. The log is where those values live.

Worked example: The logging matrix

	force	**no-force**
no-steal	neither undo nor redo	redo only
steal	undo only	undo and redo

Steal ⇒ undo; no-force ⇒ redo. No-steal + force needs no log and is unusable: no-steal pins every dirty page of every running transaction (a large update exhausts the pool), force makes commit wait for hundreds of random writes. Real systems choose **steal + no-force**, the fastest cell, and pay for it with a **write-ahead log** holding both.

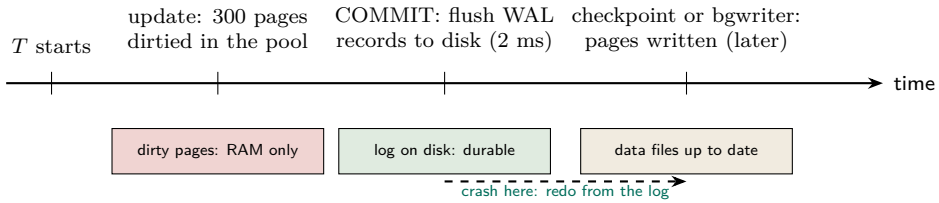
4.3 The write-ahead rule and what commit really writes

Mechanism: Two rules

The log is an append-only file of records "transaction T changed page p from old to new".

1. **Log before data.** Before a dirty page is written to disk, all log records describing its changes are already on disk. Any change found on disk after a crash can then be undone.
2. **Log forced at commit.** At commit the log up to T's commit record is flushed (`fsync`). The data pages stay dirty in the pool. Any committed change missing from the data files after a crash can then be redone.

Commit costs *one sequential append* of a few kilobytes, not hundreds of random page writes.



Worked example: Mock v2 D3: "COMMIT takes 2 ms but changed 300 pages. How is that safe?"

At `COMMIT`, PostgreSQL writes and `fsync`s only the transaction's WAL records, a sequential append of a few KB. The 300 dirty pages (300×8 KB $= 2.4$ MB at random positions) stay in `shared_buffers` and are written later by the background writer or the next checkpoint: that is no-force, and it is what makes 2 ms possible; a forced commit would wait for 300 random writes, 3 s on HDD at 10 ms each, 30 ms on SSD. After a power cut, recovery starts at the last checkpoint and replays the log: every change of a transaction whose commit record is there is **redone** onto the data pages, so the committed transaction loses nothing. A transaction with no commit record is treated as aborted: any page of it that was stolen to disk is **undone**, or, in PostgreSQL, its tuple versions are simply never visible, so it leaves no trace.

Mock drill

Mock v2 A8: steal + no-force needs undo and redo (answer c), one line: "steal lets uncommitted pages reach disk, no-force lets committed pages stay in memory". **Mock v2 D3** is the example above: claim (only the log is written), mechanism (WAL forced at commit, pages later), number (2.4 MB, 3 s forced on HDD against 2 ms), consequence (redo for committed, nothing for uncommitted). **Mock 1, 2.6**: the matrix with one sentence per cell. Full chapter: §6.2–6.3 pages 13–15.

In the real world. PostgreSQL's WAL carries redo records only. MVCC never overwrites a row in place, so an uncommitted transaction's new versions sit beside the old ones and are hidden by the commit status; there is nothing to undo. The matrix is the general answer; PostgreSQL is the case where undo is free.

5 Sizing the Pool

Trade-offs: Why the rule of thumb is 25% of RAM, not 80%

A DBA with 64 GB proposes `shared_buffers = 52GB` because "more cache is always better". Three things go wrong.

- **The OS page cache starves.** PostgreSQL relies on it for everything that is not a pool hit: WAL writes, temporary files, the "read" pages of §2. With 12 GB left, pool misses become disk reads.
- **Sorts and hash joins have no room.** `work_mem` is allocated per operator per connection *outside* the pool; 100 connections running a 64 MB sort each need 6.4 GB that no longer exists: swapping, external sorts, multi-batch hashes.
- **Checkpoints become storms.** More frames mean more dirty pages between checkpoints, so each checkpoint writes tens of gigabytes in a burst; latency spikes every few minutes.

Measure before deciding: the hit ratio (`pg_statio_user_tables`) and the usage counts in `pg_buffercache` (mostly 4–5: the pool could use more; mostly 0–1: it already exceeds the working set). Start at 25% (16 GB here), raise only while the hit ratio responds, stop near 40%.

Mock drill

Mock v2 D1 is this box in four lines: claim (no), mechanism (OS cache, `work_mem` outside the pool, checkpoints), number (6.4 GB for 100 sorts), consequence (swapping, spills, stalls), then what to measure. Full chapter: §7.4 page 17.

Warning / Gotcha: The three mistakes that cost points

- "`read` means it came from the disk." It means not in `shared_buffers`; the OS cache may have served it. Use the timing to tell.
- Tracing LRU as FIFO. A hit renews the page under LRU and changes nothing under FIFO. Belady's anomaly belongs to FIFO.
- "No-force means committed data can be lost." The *pages* need not be written at commit; the *log* is, and redo restores the pages.

Summary: The essential in four lines

1. EAT $= hT_{\text{hit}} + (1-h)T_{\text{miss}}$: the miss ratio rules; 99% on HDD is still 100 μs per access, 98% on SSD is 2.1 μs.
2. `Buffers`: `hit` in the pool, `read` not in the pool (OS cache or disk), `temp` is a spill. A 1.3× cold/warm gap means the OS cache served the reads.
3. LRU renews on a hit, FIFO does not; FIFO has Belady's anomaly, LRU cannot. Clock-sweep approximates LRU with `usage_count` 0–5. A scan of a table above `shared_buffers`/4 runs through a 256 KB ring, so it cannot flood the pool.
4. Steal needs undo, no-force needs redo; real systems run steal + no-force with a write-ahead log (log before data, forced at commit): commit is one sequential append, recovery redoes committed work. Size the pool at 25% of RAM; 80% starves the OS cache, `work_mem` and the checkpointer.